Financial Transaction Category Classification Using Machine Learning

Brian Ly
Bryce Rambach
Kien Tu

1. Abstract

This project aims to build and compare machine learning models that classify financial transactions into spending categories. We will preprocess two Kaggle datasets to handle missing values, duplicates, text encoding, feature scaling, and class imbalance. Three models, Random Forest, Support Vector Machine, and a Multi-Layer Perceptron, will each be trained, tuned, and evaluated using accuracy, precision, recall, and F1-score. The goal is to determine which approach best balances classification accuracy with practical reliability for real-world financial data.

2. Introduction

Sorting financial transactions into categories like Groceries, Entertainment, or Utilities is a core part of personal budgeting. Banks and finance apps do this automatically, but the underlying classification task is far from trivial when transaction descriptions are messy, abbreviated, or inconsistent across merchants.

Our datasets include attributes such as date, transaction description, category, amount, type, UserID, and account number. The transaction description field is the richest signal for classification since it often names the merchant or service directly. Features like transaction amount can also reveal user-level spending patterns that help distinguish categories.

We are using two financial datasets from Kaggle: a Personal Finance dataset and a Personal Transaction dataset. Both contain real-world financial activity and together provide a larger, more varied training set than either would alone.

The objective is to implement and compare three machine learning models that classify financial transactions into their correct spending categories, and to identify which model performs best across our chosen evaluation metrics.

3. Methodology

3.1 Data Preprocessing

We will use both the Personal Finance Data and the Personal Transaction Data datasets from Kaggle. Because these come from different sources, we will need to standardize column names, unify category labels, and merge them into one consolidated dataset before training. This gives our models more training examples and a wider variety of transaction types.

Handling Missing Values: We will start with an exploratory analysis to find missing or incomplete entries. For numerical features like transaction amount, we will use mean or median imputation depending on the data distribution. For categorical features like transaction description, we will either fill in the mode value or drop the row if missing data represents a small fraction of the dataset. Financial data tends to have occasional gaps from recording errors, and dropping every incomplete row would cost us too many training samples.

Removing Duplicate Records: Financial datasets often contain duplicate entries from system errors or repeated logging. We will check for rows with identical values across date, amount, description, and UserID, then remove them so the models are not biased by repeated data points.

Feature Extraction and Encoding: Transaction descriptions contain unstructured text that our models cannot use directly. We will apply TF-IDF (Term Frequency-Inverse Document Frequency) vectorization to convert these descriptions into numerical feature vectors. TF-IDF captures how important a given word is within the full set of transactions. A word like "Netflix" would carry a strong signal for the Entertainment category, while a common word like "payment" would be downweighted. For other categorical features like transaction type (debit vs. credit), we will use one-hot encoding. The target variable (transaction category) will be label-encoded into integers.

Feature Scaling: We will standardize all numerical features using z-score normalization so that each feature has a mean of zero and standard deviation of one. This step is necessary for our SVM and MLP models, which are both sensitive to the relative scale of input features. Without scaling, a feature like transaction amount (ranging from $1 to $10,000) would dominate features with smaller numeric ranges.

Handling Imbalanced Data: Financial transaction datasets tend to have uneven class distributions. Categories like Food or Shopping often have far more entries than categories like Medical or Insurance. We will analyze the class distribution after merging and, if we find significant imbalance, apply SMOTE (Synthetic Minority Over-sampling Technique). SMOTE generates new synthetic examples by interpolating between existing minority-class samples rather than duplicating them, which reduces the risk of overfitting compared to simple oversampling.

3.2 Model Selection

We selected three supervised learning models for this multi-class classification task. Each one takes a different approach to drawing decision boundaries, which will let us make a meaningful comparison of their strengths and weaknesses on financial data.

Model 1: Random Forest

Random Forest builds many decision trees during training and classifies each transaction by majority vote across all trees. We chose it for three reasons. First, it handles mixed feature types

(numerical and categorical) without much extra preprocessing. Second, averaging across many trees reduces variance and makes it less prone to overfitting than a single decision tree. Third, it produces feature importance scores, so we can see which inputs (description terms, amount, transaction type) contribute most to correct classifications. We will tune the number of trees, maximum tree depth, and minimum samples per leaf using grid search with cross-validation.

Model 2: Support Vector Machine (SVM)

SVM finds the optimal separating hyperplane between classes in high-dimensional feature space. For multi-class classification we will use a one-vs-rest strategy. SVM is a strong fit here because TF-IDF vectorization of transaction descriptions produces a large number of features, and SVM performs well in high-dimensional settings. Financial transaction categories also tend to have distinct textual patterns that SVM can exploit through clear margin separation. We will test both linear and RBF (Radial Basis Function) kernels and tune the regularization parameter C and the RBF gamma parameter via grid search with cross-validation.

Model 3: Multi-Layer Perceptron (MLP)

The MLP is a feedforward neural network with an input layer, hidden layers, and an output layer. We chose it because it can capture non-linear relationships between features and categories that Random Forest and SVM may miss, making it a useful contrast in our comparison. We plan to use two hidden layers with ReLU activations and a softmax output layer for multi-class probabilities. We will tune the number of neurons per layer, learning rate, batch size, and number of training epochs using grid search or randomized search with cross-validation. We will also use early stopping, monitoring validation loss to halt training before the model overfits.

All three models will be implemented in Python using scikit-learn, with the option of TensorFlow or Keras for the MLP if we need more architectural flexibility. The merged dataset will be split 80/20 into training and testing sets using stratified sampling to preserve the original class distribution. All preprocessing, including TF-IDF fitting and feature scaling, will be fit on the training set only and then applied to the test set to prevent data leakage.

3.3 Evaluation Metrics

We will evaluate all three models using accuracy, precision, recall, F1-score, confusion matrices, and runtime. Each metric captures a different aspect of classifier performance, and together they give a complete picture.

Accuracy (overall and per-class) measures the share of transactions classified correctly. Overall accuracy is a useful baseline, but it can be misleading when some categories have far more examples than others. Per-class accuracy shows where each model does well and where it struggles.

Precision measures, for each category, how many of the transactions the model labeled as that category actually belong there. High precision means few false positives. In financial classification, a false positive means a transaction gets assigned to the wrong budget category, which would distort a user's spending breakdown.

Recall measures how many of the transactions that actually belong to a category the model successfully finds. High recall means few missed transactions. This matters most for smaller categories where missing even a few entries is noticeable.

F1-Score is the harmonic mean of precision and recall. Because mislabeling a financial transaction has direct consequences for a user's budget tracking, we want a metric that penalizes models that sacrifice one for the other. We will use F1 as our primary optimization target. We will report both per-class F1-scores and a macro-averaged F1 across all categories.

Confusion Matrix: We will generate a confusion matrix for each model to visualize which categories get mixed up. For example, if "Dining" transactions are often misclassified as "Groceries," the confusion matrix will show that pattern and point us toward targeted improvements.

Runtime Analysis: We will record training time and prediction time for each model. A model that is slightly less accurate but trains in seconds rather than minutes could be the better practical choice for a real-world application.

To rank the three models against each other, we will use a weighted composite score: 0.4(F1) + 0.25(Precision) + 0.25(Recall) + 0.1(Accuracy). This weighting reflects our priority: F1-score carries the most weight because it balances precision and recall, precision and recall are weighted separately to reward models that perform well on both dimensions independently, and accuracy serves as a final check on overall correctness.

4. Task Division

Brian Ly will implement the Random Forest model, including hyperparameter tuning and performance evaluation. Brian will also contribute to preprocessing by handling missing values and duplicate removal. For the final report, Brian will write the Results section covering Random Forest performance and contribute to the comparative analysis.

Bryce Rambach will implement the Support Vector Machine model, including kernel selection, hyperparameter tuning, and performance evaluation. Bryce will lead the feature extraction and encoding work, including TF-IDF vectorization. For the final report, Bryce will write the Methodology section and contribute to the comparative analysis.

Kien Tu will implement the Multi-Layer Perceptron neural network, including architecture design, hyperparameter tuning, and performance evaluation. Kien will handle feature scaling and class imbalance analysis (SMOTE). For the final report, Kien will write the Introduction and Discussion sections and contribute to the comparative analysis.

All three team members will collaborate on dataset merging, final model comparison, and report editing. Each member will participate in both coding and report writing.